To *really* create a language you need to be able to modify the syntax. Although the internal DSL-making potential of CoffeeScript means you can create mini-languages for some tasks, you're still fundamentally constrained to what CoffeeScript syntax accepts.

In order to stretch the language itself, you need to be able to modify it. For a compiled language like CoffeeScript, that means you need to intercept the compiler somehow. To do this you can either work with the compiler at the token or AST stage, or you can place something before or after the compiler. The easiest (though not the most sensible) way to do this is to just preprocess source code using `eval`.

8.4.1. Can you just eval?

JavaScript has an `eval` function that takes a string of JavaScript code and executes it in the running program. The string of code is evaluated as if you had written it directly into the program. For example, `var` declarations will go into the current scope. Try it on the Node JavaScript REPL:

```
node> eval('var x = 2');
node> x
node> # 2
```

The use of `eval` is considered dangerous because the code is executed with the privileges of the caller. This means that anything you `eval` can do anything that can be done at the point the `eval` was invoked. The `eval` function is dangerous, but it's certainly interesting.

The CoffeeScript compiler also has an `eval` method that compiles and evaluates a string of program code. The difference with the `eval` provided by the CoffeeScript compiler (apart from the fact that it evaluates CoffeeScript) is that by default it sandboxes the code that it evaluates by running it in a separate context. The evaluation of the `eval` call is the evaluation of the code contained in the string:

```
coffee = require 'coffee-script'
coffee.eval '2'
# 2
```